LAPORAN UTS

KLASIFIKASI BINER INDIKASI KEBAKARAN PADA CITRA D-FIRE

PEMBELAJARAN MENDALAM

Oleh:

Kelompok: 6

Anggota:

Julian Caesar Eben Ezer Simorangkir (231111575)

Gloria Apriyanti Siagian (231111304)

Howard Farel Hura (231111644)

Cindy Nathania Perangin Angin (231111567)

Kevin Luther Kiyosaki Gultom (231111131)



PROGRAM STUDI S-1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA
UNIVERSITAS MIKROSKIL
MEDAN

SEMESTER GANJIL

TA. 2026-2027

DAFTAR ISI

DAFTAR GAMBAR 3
DAFTAR TABEL 4
Pembagian Tugas 5
BAGIAN A. LAPORAN TEKNIS PROYEK 6
1. Kompleksitas Masalah 6
2. Dataset yang Digunakan 6
3. Exploratory Data Analysis (EDA) 7
4. Preprocessing Data 8
5. Model yang Digunakan 9
6. Hasil Evaluasi 13
7. Teknologi yang Digunakan 16
8. Deployment dan Demo 17
9. Lampiran 23
10. Daftar Pustaka 26
BAGIAN B. TAUTAN 27

DAFTAR GAMBAR

Gambar 1. Distribusi label pada data latih, validasi, dan uji 7
Gambar 2. Dua citra uji tanpa indikasi yang dikenali dengan benar [6] 8
Gambar 3. Dua citra uji berindikasi yang dikenali dengan benar [6] 8
Gambar 4. Arsitektur MobileNetV2 dan kepala deteksi 10
Gambar 5. Empat peta kontras lokal pada foto kebakaran 11
Gambar 6. Confusion matrix pada 4.306 citra uji dengan ambang 0,20 13
Gambar 7. Loss total, klasifikasi, dan koordinat selama 29 epoch pelatihan 14
Gambar 8. Dua citra negatif yang memicu peringatan asap 15
Gambar 9. Dua citra beranotasi positif yang diprediksi tanpa indikasi 15
Gambar 10. Antarmuka web dengan tiga foto testing yang masih menunggu prediksi 17
Gambar 11. Prediksi benar pada foto kebakaran: api 0,337 dan asap 0,473 18
Gambar 12. Prediksi salah pada foto kucing: asap 0,216 melewati ambang 0,20 19
Gambar 13. Union OFF menampilkan 11 kotak hasil penyaringan secara terpisah 20
Gambar 14. Berkas model, anchor, dan manifest yang disertakan dalam repository 21
Gambar 15. Dependensi ONNX Runtime Web dan perintah pengembangan pada package.json 22
Gambar 16. Konfigurasi direktori publik dan tipe berkas pada vercel.json 22
Gambar 17. Deployment produksi berstatus Ready dan terhubung ke branch main 23
Gambar 18. Kurva asli loss total, loss kelas, dan loss kotak pada notebook 25
Gambar 19. Matriks kebingungan asli pada 4.306 citra uji 25

DAFTAR TABEL

Tabel 1. Pembagian Tugas Kelompok 6 5
Tabel 2. Jumlah citra dan kotak anotasi pada setiap bagian data [6] 7
Tabel 3. Komposisi citra D-Fire menurut objek yang dianotasi [2] 7
Tabel 4. Bentuk tensor pada komponen detektor 10
Tabel 5. Pelatihan dan hasil validasi per tahap 11
Tabel 6. Pengaruh ambang keputusan pada tiga foto demonstrasi 12
Tabel 7. Perbandingan ambang pada model lanjutan dan data uji yang sama 12
Tabel 8. Kesesuaian keluaran model Keras dan hasil ekspor ONNX 13
Tabel 9. Metrik biner untuk kelas positif “ada indikasi” 14
Tabel 10. Perangkat lunak eksperimen dan kegunaannya 16

Pembagian Tugas

NIM	Nama Anggota	Peran	Rincian Pekerjaan
231111575	Julian Caesar Eben Ezer Simorangkir	Ketua dan koordinasi laporan	Mengoordinasikan pembagian kerja dan kelengkapan pengumpulan; menyusun latar belakang serta kompleksitas masalah; menyatukan bagian laporan; memeriksa kesesuaian laporan, repository, dan aplikasi.
231111304	Gloria Apriyanti Siagian	Dataset dan EDA	Menelusuri sumber D-Fire dan format anotasi; merangkum jumlah citra, kelas, dan pembagian data; menyusun distribusi label serta contoh citra; menjelaskan temuan EDA dan kaitannya dengan preprocessing.
231111644	Howard Farel Hura	Web app dan deployment	Menyiapkan antarmuka unggah dan antrean foto; mengintegrasikan model ONNX beserta preprocessing dan penyaringan kotak; mengatur union serta ekspor hasil; menghubungkan GitHub dengan Vercel dan menguji aplikasi publik.
231111567	Cindy Nathania Perangin Angin	Evaluasi dan dokumentasi	Memeriksa confusion matrix dan perhitungan metrik; membandingkan ambang skor; membahas prediksi benar, false positive, dan false negative; menata grafik, tabel, sitasi, serta dokumentasi pengujian aplikasi.
231111131	Kevin Luther Kiyosaki Gultom	Notebook dan model CNN	Menyusun UTS-Notebook dan alur pengolahan citra; membangun MobileNetV2, cabang kontras, anchor, serta kepala deteksi; mengatur transfer learning dan fine-tuning; menyimpan model, mengevaluasi keluaran, dan memeriksa kesesuaian ekspor ONNX.

Tabel 1. Pembagian Tugas Kelompok 6

Program Studi S-1 Teknik Informatika, Fakultas Informatika, Universitas Mikroskil.

BAGIAN A. LAPORAN TEKNIS PROYEK

1. Kompleksitas Masalah

Proyek ini menentukan apakah pada sebuah foto terlihat indikasi kebakaran. Asap saja, api saja, atau keduanya menghasilkan label “ada indikasi”. Untuk memperoleh keputusan itu, model terlebih dahulu belajar mengenali lokasi asap dan api dari kotak anotasi D-Fire.

Mengapa masalah ini dianggap kompleks?

1. Ukuran dan bentuk objek berubah antarfoto. Api di kejauhan bisa tampak sebagai titik terang, sedangkan asap dapat menyebar tanpa batas yang tegas.
2. Objek lain dapat menyerupai indikasi kebakaran. Contohnya, dua foto langit berawan pada data uji memicu prediksi asap meskipun keduanya tidak memiliki anotasi.
3. Satu foto dapat memuat beberapa objek. Model harus belajar menentukan kelas serta posisi tiap objek, sedangkan evaluasi utama menghitung satu keputusan untuk seluruh foto.

Apa tantangan utama dalam menyelesaikan masalah ini?

Saat ukuran citra diperkecil, sebagian detail asap tipis atau api yang jauh dapat hilang. Di sisi lain, satu kotak yang keliru pada awan sudah cukup untuk memicu peringatan. Ambang skor perlu mempertimbangkan kedua kesalahan tersebut. Analisis pada bagian hasil membedakan foto berindikasi yang terlewat dari foto tanpa indikasi yang memicu peringatan.

2. Dataset yang Digunakan

Sumber dan jenis dataset

D-Fire berisi foto dunia nyata yang dianotasi untuk mendeteksi asap dan api [1, 2]. Eksperimen menggunakan arsip versi 1 dari mirror Kaggle yang dirujuk oleh penerbit [3]. Arsip tersebut sudah membagi citra menjadi data latih, validasi, dan uji.

Jumlah data, kelas, dan format anotasi

Dataset memuat 21.527 citra JPG. Setiap citra berpasangan dengan berkas TXT bernama dasar sama. Satu baris anotasi berisi ID kelas (0 untuk asap, 1 untuk api), koordinat pusat kotak, lebar, dan tinggi relatif terhadap ukuran gambar. Citra dengan berkas label kosong termasuk kelas “tanpa indikasi”.

Bagian data	Jumlah citra	Ada indikasi	Tanpa indikasi	Jumlah kotak
Latih	14.122	7.664	6.458	17.432
Validasi	3.099	1.724	1.375	3.932
Uji	4.306	2.301	2.005	5.193
Total	21.527	11.689	9.838	26.557

Tabel 2. Jumlah citra dan kotak anotasi pada setiap bagian data [6]

Seberapa kompleks dataset tersebut?

Satu kotak mewakili satu objek. Karena satu citra dapat memiliki beberapa kotak, 11.689 citra positif menghasilkan 26.557 kotak anotasi. Menurut penerbit D-Fire, 14.692 kotak berlabel api dan 11.865 kotak berlabel asap [2].

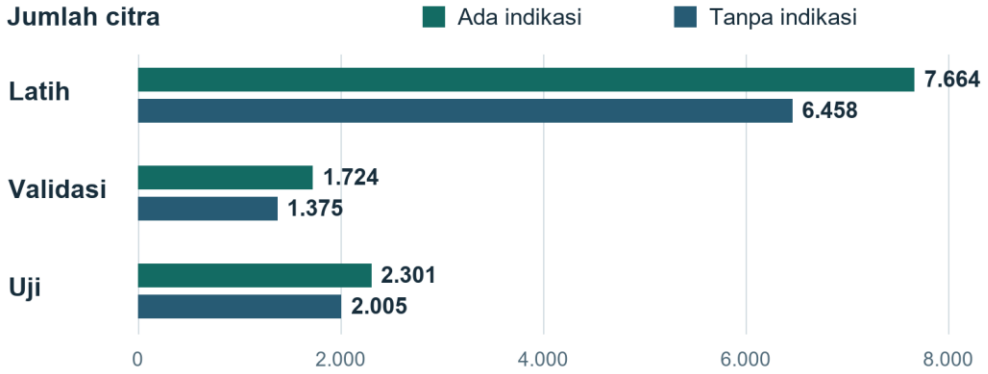
Isi anotasi pada satu citra	Jumlah citra
Hanya asap	5.867
Hanya api	1.164
Asap dan api	4.658
Tanpa asap dan api	9.838

Tabel 3. Komposisi citra D-Fire menurut objek yang dianotasi [2]

Foto yang hanya mengandung asap lebih banyak daripada foto yang hanya mengandung api. Sebaliknya, jika setiap kotak dihitung, anotasi api lebih banyak. Oleh karena itu, distribusi objek dan distribusi citra dibaca secara terpisah.

3. Exploratory Data Analysis (EDA)

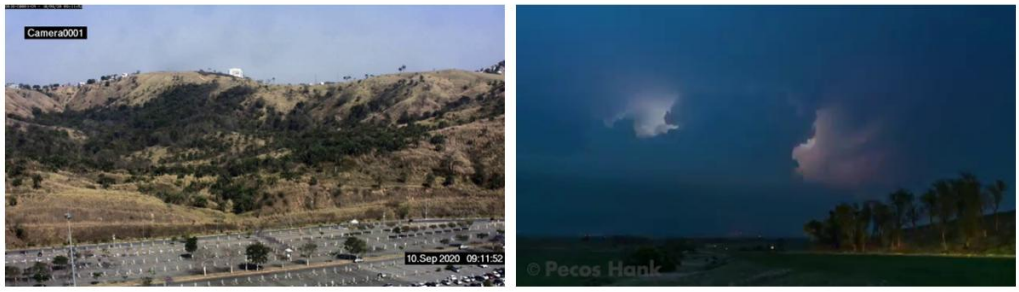
Distribusi label citra



Gambar 1. Distribusi citra dengan dan tanpa indikasi pada data latih, validasi, dan uji [6]

Sebanyak 54,3% citra memiliki setidaknya satu anotasi. Proporsi positif pada data latih (54,3%), validasi (55,6%), dan uji (53,4%) relatif berdekatan. Ketidakseimbangan lebih terasa di dalam satu citra karena sebagian besar calon kotak jatuh pada latar. Pada saat pelatihan, focal loss mengurangi bobot kandidat latar yang sudah mudah dikenali.

Contoh citra dari tiap kelas



Gambar 2. Dua citra uji tanpa indikasi yang dikenali dengan benar [6]



Gambar 3. Dua citra uji berindikasi yang dikenali dengan benar [6]

Gambar 2 memperlihatkan contoh latar luar ruang tanpa anotasi. Pada Gambar 3, model menandai api kecil pada adegan malam dan asap yang menyebar pada adegan terang. Kotak berwarna pada kedua foto menunjukkan prediksi model.

Temuan yang memengaruhi pengolahan citra

Contoh tersebut memperlihatkan perbedaan skala objek dan pencahayaan. Pengolahan citra memakai ukuran tetap agar foto dapat diproses bersama dalam satu batch. Untuk mempertahankan informasi setempat, detektor memadukan fitur spasial MobileNetV2 dengan respons terang dan gelap dari dua ukuran lingkungan piksel.

4. Preprocessing Data

Bagaimana citra dan label diperiksa?

1. Kode memeriksa pasangan JPG dan TXT serta jumlah gambar pada setiap bagian data. Jika arsip diunduh, SHA-256 digunakan untuk memeriksa kesesuaian berkas.
2. Baris kosong dilewati. Untuk setiap baris berisi anotasi, kode memeriksa ID kelas dan memastikan lebar serta tinggi kotak tidak negatif. Berkas TXT kosong tetap dipakai sebagai contoh negatif. Berkas label yang hilang menyebabkan pemeriksaan pasangan berkas gagal.
3. Pusat dan ukuran kotak diubah menjadi koordinat dua sudut. Nilainya dibatasi pada rentang 0 sampai 1. Jika lebar atau tinggi menjadi nol, komponen ukuran tersebut

dikeluarkan dari perhitungan galat regresi. Pemeriksaan duplikat berdasarkan isi foto dan koreksi label manual tidak tercatat pada notebook.

Ukuran citra dan normalisasi piksel

Pillow membaca setiap citra sebagai RGB, mengubah ukurannya secara bilinear menjadi 192 piksel lebar dan 128 piksel tinggi, lalu menyusun nilainya sebagai float32. Urutan masukan model adalah 128 × 192 × 3. Pengubahan langsung ke ukuran tetap dapat mengubah rasio aspek gambar.

Cabang MobileNetV2 mengubah piksel x menjadi $x/127{,}5 - 1$ agar nilainya berada dalam rentang −1 sampai 1. Cabang kontras membagi piksel dengan 255 sebelum menghitung luminans. Data uji dan foto demo menjalani transformasi yang sama.

Penyusunan target deteksi

Citra yang telah diubah ukurannya digunakan untuk menyusun target deteksi. Kode menghitung Intersection over Union (IoU), yaitu luas irisan dibagi luas gabungan, antara kotak awal (anchor) dan kotak anotasi.

Anchor dengan IoU minimal 0,50 menjadi positif. Anchor dengan IoU 0,40 hingga kurang dari 0,50 diabaikan dan sisanya menjadi latar. Selain aturan ambang tersebut, setiap objek diberi satu anchor agar objek kecil tetap menyumbang target pelatihan.

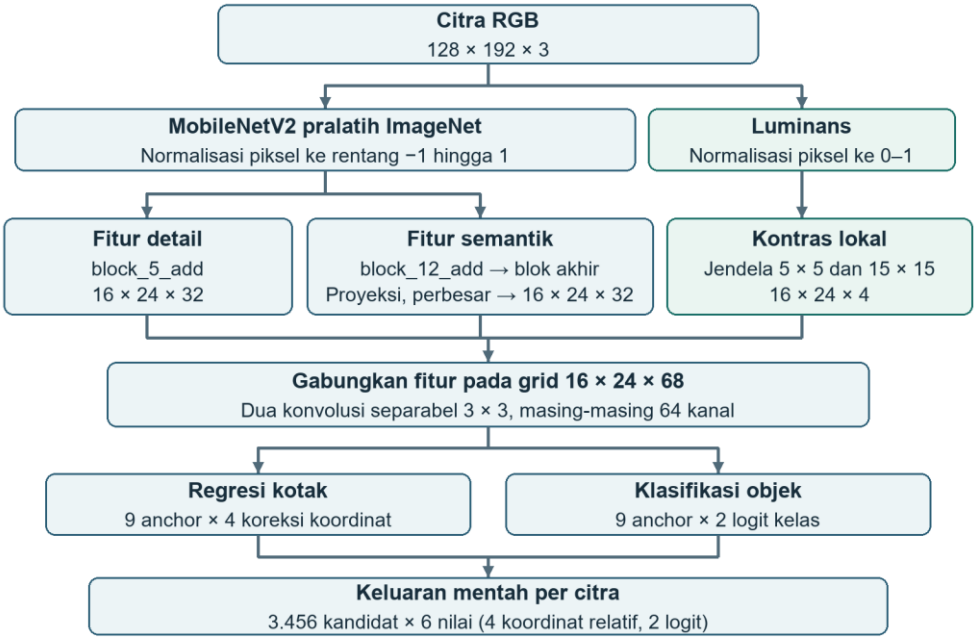
Pembagian data dan pencegahan kebocoran

Pembagian bawaan terdiri dari 65,6% data latih, 14,4% validasi, dan 20,0% uji. Hanya data latih yang diacak dan digunakan untuk memperbarui bobot. Loss validasi menentukan bobot yang dipertahankan. Data uji digunakan untuk evaluasi akhir. Notebook mencatat bahwa bagian uji ini pernah dipakai pada percobaan terdahulu, sehingga generalisasi ke kumpulan foto baru masih perlu diuji secara terpisah.

5. Model yang Digunakan

Jenis model dan alasan pemilihan

Kami menggunakan MobileNetV2 dengan bobot awal ImageNet sebagai ekstraktor fitur [4]. Satu foto D-Fire dapat memuat asap, api, atau keduanya di lokasi berbeda. Oleh karena itu, kepala klasifikasi ImageNet diganti dengan kepala deteksi yang memperkirakan kelas dan posisi objek. Pendekatan ini juga memberi lokasi yang dapat diperiksa ketika sebuah foto memicu peringatan. Konvolusi depthwise pada MobileNetV2 menekan beban perhitungan [4], sedangkan bobot awal menyediakan fitur visual untuk disesuaikan dengan citra kebakaran. Bukti eksperimen dalam proyek ini berupa penurunan loss validasi dari tahap transfer ke fine-tuning pada Tabel 5.



Gambar 4. Arsitektur MobileNetV2 dan kepala deteksi asap serta api

Arsitektur dan fungsi tiap lapisan

Model menerima citra RGB berukuran 128 × 192 × 3 (tinggi × lebar × kanal) dan memiliki 2.312.253 parameter. Dua jalur membaca citra yang sama. MobileNetV2 mempelajari pola visual, sedangkan jalur kontras menghitung perbedaan terang–gelap di sekitar piksel. Backbone memuat 35 lapisan Conv2D dan 17 DepthwiseConv2D. Kepala deteksi menambahkan satu proyeksi Conv2D, dua SeparableConv2D, dan dua Conv2D keluaran.

Komponen	Operasi dan keluaran
Fitur MobileNetV2	block_5_add menghasilkan fitur detail 16 × 24 × 32. Fitur tengah block_12_add berukuran 8 × 12 × 96 dan diteruskan ke blok akhir.
Penggabungan	Fitur akhir diproyeksikan menjadi 32 kanal dan diperbesar 4 kali. Penggabungan dengan fitur detail dan 4 kanal kontras menghasilkan tensor 16 × 24 × 68.
Kepala deteksi	Dua konvolusi separabel 3 × 3, masing-masing dengan 64 kanal dan aktivasi ReLU. Dua konvolusi 1 × 1 menghasilkan 36 kanal lokasi dan 18 kanal kelas.
Keluaran per citra	Grid 16 × 24 dengan 9 kotak awal (anchor) per posisi menghasilkan 3.456 kandidat. Setiap kandidat memuat 4 koreksi koordinat dan 2 logit kelas.

Tabel 4. Bentuk tensor pada komponen detektor

Konfigurasi dan tahapan pelatihan

Pada tahap pertama, bobot MobileNetV2 dibekukan agar kepala deteksi dapat belajar menghubungkan fitur dengan anotasi D-Fire. Pada dua tahap berikutnya, 30 lapisan terakhir MobileNetV2 dibuka. Semuanya merupakan tahap berurutan pada satu model.

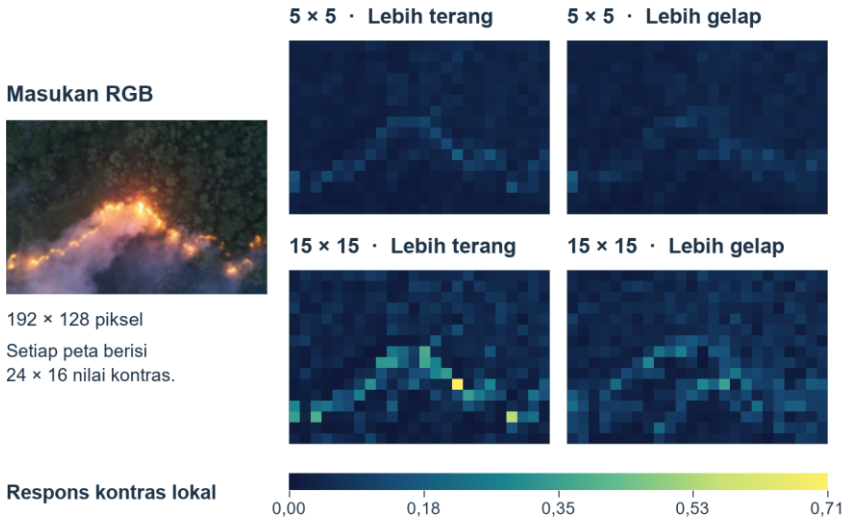
Tahap	Epoch	Laju belajar awal	Bagian yang dilatih	Loss validasi terbaik
Transfer	1–6	0,001	Kepala deteksi	1,3522
Fine-tuning	7–21	0,00001	Kepala dan 30 lapisan akhir	1,2583
Fine-tuning lanjutan	22–29	0,000005	Kepala dan 30 lapisan akhir	1,2488

Tabel 5. Pelatihan dan hasil validasi per tahap

1. Optimasi memakai Adam, batch 16, seed 42, dan batas norm gradien 5. Statistik Batch Normalization dipertahankan pada mode inferensi.
2. Focal loss memberi bobot lebih kecil pada prediksi kelas yang sudah mudah, termasuk banyak kandidat latar [5]. Model memakai $\gamma = 2$, bobot 0,25 untuk target positif dan 0,75 untuk target negatif pada setiap kelas.
3. Smooth L1 menghitung galat koordinat pada anchor positif. Loss kelas dan koordinat masing-masing dibagi jumlah anchor positif per citra, dengan pembagi minimum satu, kemudian dijumlahkan.
4. EarlyStopping memantau loss validasi dengan patience 3 pada transfer dan 5 pada fine-tuning. Pada tahap lanjutan, ReduceLROnPlateau membagi dua laju belajar setelah dua epoch tanpa perbaikan, dengan batas minimum 0,0000001.

Cabang kontras lokal pada dua skala

Cabang kontras menghitung luminans dari kanal RGB, lalu membandingkan tiap piksel dengan rata-rata sekitarnya pada jendela 5 × 5 dan 15 × 15. Setiap ukuran menghasilkan dua peta, satu untuk area yang lebih terang dan satu untuk area yang lebih gelap. Keempat peta diringkas pada grid 16 × 24 agar sejajar dengan fitur deteksi.



Gambar 5. Empat peta kontras lokal dari foto kebakaran pada skala intensitas yang sama

Pada foto demonstrasi, respons terang maksimum meningkat dari 0,181 pada jendela 5 × 5 menjadi 0,707 pada jendela 15 × 15. Peta kedua memperlihatkan lengkungan terang yang

lebih tegas ketika nyala dibandingkan dengan lingkungan yang lebih luas. Untuk mengukur pengaruh cabang ini terhadap akurasi diperlukan pelatihan pembanding tanpa fitur kontras.

Fitur unik 1: ambang skor untuk keputusan biner

Fitur ini mengatur kepekaan keputusan terhadap indikasi yang lemah. Dua logit diubah menjadi skor asap dan api melalui sigmoid, skor = 1 / (1 + exp(−logit)). Kotak valid dengan skor awal ≥ 0,01 masuk ke Non-Maximum Suppression (NMS) per kelas. NMS memilih kotak berskor lebih tinggi dan membuang kotak sekelas yang bertumpang tindih dengannya pada IoU > 0,50. Setelah memilih paling banyak 100 hasil, satu kotak dengan skor ≥ 0,20 sudah cukup untuk memberi keputusan "ada indikasi".

Foto	Skor kelas maksimum	Ambang 0,20	Ambang 0,30
Kebakaran	0,473 (asap)	Ada	Ada
Pohon	0,106 (api)	Tidak ada	Tidak ada
Kucing	0,216 (asap)	Ada	Tidak ada

Tabel 6. Pengaruh ambang keputusan pada tiga foto demonstrasi

Skor maksimum pada panel dihitung dari seluruh anchor sebelum penyaringan. Keputusan biner menggunakan kotak yang lolos seluruh tahap. Pada foto kucing, deteksi asap lolos pada ambang 0,20 dan gugur pada 0,30. Perbandingan tiga foto ini merupakan simulasi pada keluaran yang sama; aplikasi publik memakai ambang tetap 0,20.

Bukti pada seluruh 4.306 citra uji tersedia dalam perbandingan ambang pada bobot model lanjutan yang sama [6]. Ambang 0,20 mengurangi indikasi yang terlewat dari 791 menjadi 280, tetapi menambah alarm salah dari 18 menjadi 129. Pertukaran ini menjadi alasan penggunaan ambang 0,20.

Ambang	Akurasi	Precision	Recall	F1-score	FP / FN
0,35	81,21%	98,82%	65,62%	78,87%	18 / 791
0,20	90,50%	94,00%	87,83%	90,81%	129 / 280

Tabel 7. Perbandingan ambang pada model lanjutan dan data uji yang sama

Fitur unik 2: inferensi ONNX di browser

Inferensi adalah penggunaan parameter hasil pelatihan untuk menghitung prediksi pada foto baru. ONNX (Open Neural Network Exchange) menyimpan model sebagai graf operasi, yaitu urutan perhitungan yang saling terhubung, beserta parameter numerik seperti bobot dan bias serta bentuk masukan dan keluaran [8]. Bobot tetap selama prediksi berlangsung.

Pada proyek ini, tools/export_model.py memuat model Keras lalu mengekspornya menjadi model.onnx bertipe float32 dengan opset 17, yaitu versi operasi ONNX yang digunakan. ONNX Runtime Web menjalankan graf tersebut melalui WebAssembly pada CPU perangkat pengguna [9]. Pemanggilan session.run() dilakukan di Web Worker, konteks kerja terpisah dari halaman utama, sehingga antarmuka dapat menangani interaksi selama perhitungan.

Fitur ini memungkinkan model digunakan melalui browser: Vercel menyajikan berkas aplikasi dan model, sedangkan foto diproses pada perangkat pengguna. Model, anchor, dan runtime dimuat saat prediksi pertama; sesi model digunakan kembali untuk foto berikutnya. Berkas model berukuran sekitar 9,15 MB. Pemeriksaan konversi membandingkan keluaran Keras dan ONNX pada 12 citra yang sama [6].

Pemeriksaan	Citra	Selisih skor maksimum
Keras dan ONNX CPU	12	0,00000295
Keras dan ONNX WebAssembly (Chrome)	12	0,00000332

Tabel 8. Kesesuaian keluaran model Keras dan hasil ekspor ONNX

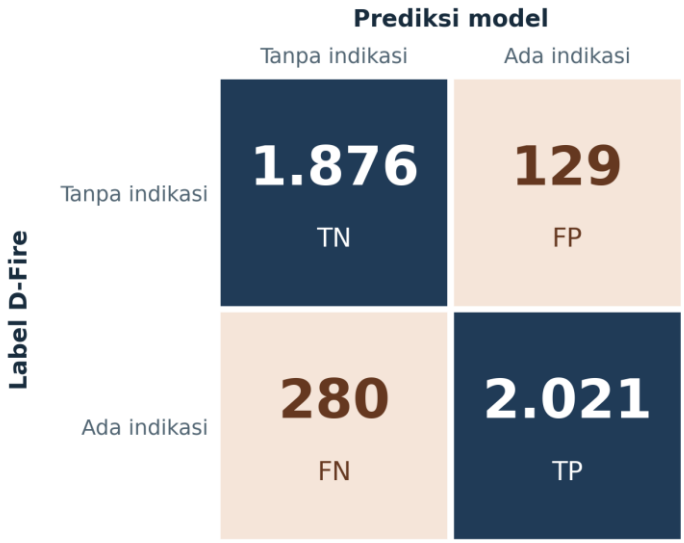
Urutan kelas dan jumlah kotak hasil deteksi sesuai pada seluruh 12 citra pembanding. Hasil ini mendukung kesesuaian ekspor pada sampel tersebut; evaluasi akurasi utama tetap berasal dari 4.306 citra test. Tangkapan layar aplikasi dan hasil pengujiannya disajikan pada bagian deployment.

6. Hasil Evaluasi

Metrik evaluasi dan confusion matrix

Pengujian mencakup 4.306 citra. Label acuan positif berarti ada sedikitnya satu kotak pada anotasi D-Fire. Dengan satu keputusan per citra, model menjawab benar pada 3.897 citra dan keliru pada 409 citra [6].

Akurasi merangkum proporsi keputusan yang benar. Precision mengukur berapa banyak peringatan yang sesuai anotasi, recall mengukur citra berindikasi yang berhasil ditemukan, dan F1-score merangkum keseimbangan precision serta recall. Keempat metrik diperlukan karena alarm salah dan indikasi yang terlewat memiliki penyebut berbeda.



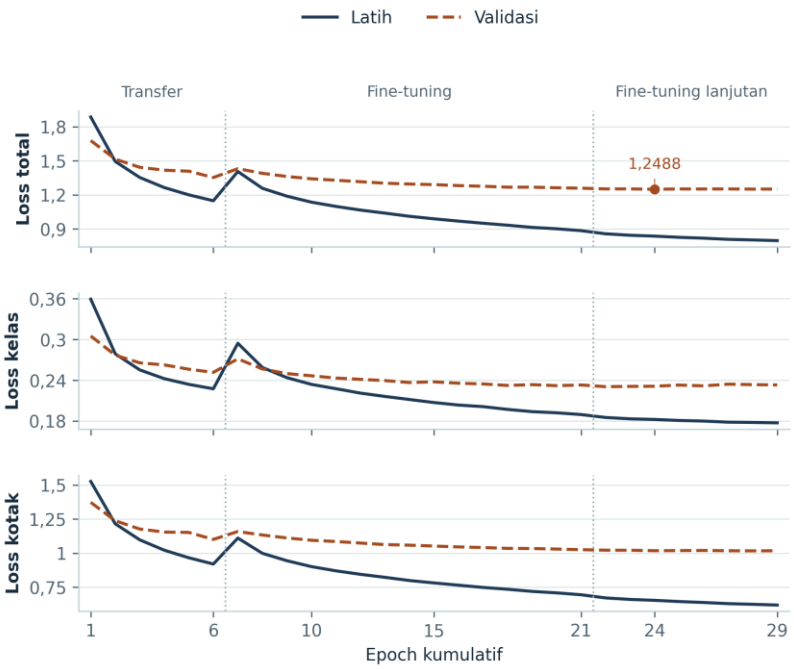
Gambar 6. Confusion matrix pada 4.306 citra uji dengan ambang 0,20

Metrik	Perhitungan	Hasil
Akurasi	(1.876 + 2.021) / 4.306	90,50%
Precision	2.021 / (2.021 + 129)	94,00%
Recall	2.021 / (2.021 + 280)	87,83%
F1-score	4.042 / (4.042 + 129 + 280)	90,81%

Tabel 9. Metrik biner untuk kelas positif "ada indikasi"

Dari 2.150 peringatan, 2.021 sesuai dengan anotasi positif (precision 94,00%). Dari 2.301 citra berindikasi, model melewatkan 280 citra (12,17%). Sebaliknya, 129 dari 2.005 citra negatif memicu peringatan (6,43%). Jadi indikasi yang terlewat lebih banyak daripada peringatan keliru pada ambang yang diuji.

Kurva pembelajaran dan perbandingan hasil



Gambar 7. Loss total, klasifikasi, dan koordinat selama 29 epoch pelatihan

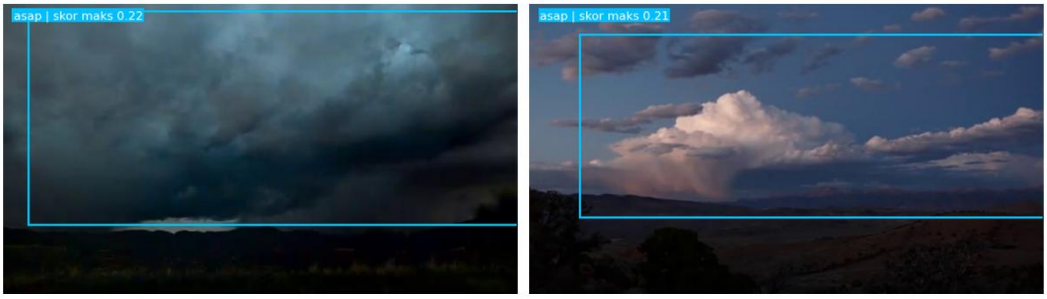
Loss validasi turun dari 1,6771 pada epoch pertama ke 1,3522 pada akhir transfer. Saat 30 lapisan terakhir dibuka, nilainya naik sementara menjadi 1,4300 lalu turun ke 1,2583 pada epoch ke-21. Nilai terendah, 1,2488, dicapai pada epoch ke-24.

Dari epoch ke-24 hingga ke-29, loss latih turun dari 0,8363 ke 0,7967. Loss validasi justru berakhir pada 1,2507. Karena lima epoch tersebut tidak memperbaiki hasil validasi, pelatihan berhenti dan bobot epoch ke-24 dipulihkan. Notebook merekam loss setiap epoch, sedangkan akurasi biner dihitung setelah penyaringan kotak pada data uji.

Loss latih yang terus menurun ketika loss validasi mulai mendatar menunjukkan gejala overfitting pada tahap akhir. Pemulihan bobot dengan loss validasi terendah membatasi

penggunaan epoch yang tidak memperbaiki validasi. Riwayat yang tersimpan tidak memuat akurasi train dan validation per epoch, sehingga grafik tersebut belum tersedia.

Analisis kesalahan prediksi



WEB09632.jpg WEB09612.jpg

Gambar 8. Dua citra negatif yang memicu peringatan asap

Pada WEB09632.jpg dan WEB09612.jpg, kotak prediksi menutupi kawasan langit berawan. Skor asap sekitar 0,22 dan 0,21, dekat dengan ambang 0,20. Tekstur awan menyerupai asap pada kedua contoh ini. Kemiripan visual tersebut merupakan kemungkinan penyebab, bukan bukti bahwa model memakai aturan khusus untuk mengenali awan.



WEB11269.jpg AoF08012.jpg

Gambar 9. Dua citra beranotasi positif yang diprediksi tanpa indikasi

Pada WEB11269.jpg dan AoF08012.jpg, tidak ada kotak yang lolos. Kedua foto memperlihatkan pemandangan dari kejauhan. Objek yang kecil setelah resize mungkin sulit dikenali. Panel menampilkan prediksi saja, sehingga jenis dan lokasi objek yang terlewat perlu diperiksa pada anotasi asli.

Kotak dalam satu kelas digabungkan untuk tampilan. Penilaian biner tetap dapat menghitung prediksi sebagai benar walaupun kotaknya salah tempat atau kelas asap dan api tertukar. Penilaian lokasi dan kelas memerlukan pencocokan dengan kotak acuan, misalnya melalui average precision per kelas. Notebook juga mencatat bahwa data uji pernah digunakan pada percobaan sebelumnya.

7. Teknologi yang Digunakan

Bahasa, library, dan lingkungan pengembangan

Eksperimen 4 November 2026 menggunakan kernel Python 3.12.4. Notebook dapat dibuka di Jupyter atau melalui ekstensi Jupyter pada VS Code. Versi pustaka yang dipakai saat pelatihan tercantum di requirements.txt [6].

Perangkat lunak	Versi	Kegunaan
Python	3.12.4	Menjalankan notebook.
TensorFlow / Keras	2.19.0 / 3.9.2	Membangun CNN, melatih model, menyaring kotak, dan menyimpan bobot.
NumPy / pandas	2.1.3 / 2.2.3	Mengolah array, target kotak dan riwayat pelatihan.
Pillow / Matplotlib	11.2.1 / 3.10.1	Membaca dan mengubah ukuran citra, menggambar kotak dan grafik.
scikit-learn	1.6.1	Menghitung akurasi dan matriks kebingungan.
SciPy / h5py / protobuf	1.15.2 / 3.13.0 / 5.29.4	Dependensi numerik, pembacaan HDF5, dan serialisasi data.

Tabel 10. Perangkat lunak eksperimen dan kegunaannya

Kami menetapkan seed 42 dan mengaktifkan operasi deterministik TensorFlow. Konfigurasi membatasi komputasi menjadi empat thread di dalam operasi dan dua thread antaroperasi. Spesifikasi GPU komputer asal tidak tercatat. README memperkirakan kebutuhan ruang kosong sekitar 15 GB untuk satu kali pelatihan.

Antarmuka dan lingkungan aplikasi

Aplikasi publik dibangun dengan HTML, CSS, dan JavaScript berbentuk modul. index.html menyusun halaman, styles.css mengatur tampilannya, dan app.js menangani antrean foto, tombol, konfigurasi union, serta hasil. engine.worker.js memuat model dan menjalankan inferensi; preprocess.js melakukan resize RGB; postprocess.js menghitung skor, menyaring kotak, dan membentuk label biner [6].

ONNX Runtime Web 1.31.1 beserta WebAssembly disertakan di dist/vendor/. Ekspor model memakai tf2onnx 1.17.0. Versi dependensi web dicatat pada package.json dan package-lock.json, sedangkan dependensi ekspor tercantum dalam tools/export-requirements.txt.

Platform hosting yang digunakan adalah Vercel, terhubung ke branch main pada repository GitHub. Folder dist/ menjadi direktori publik. Pengguna dapat memasukkan maksimal 25 foto JPG, PNG, atau WebP; batas per foto adalah 20 MiB dan 40 megapiksel.

8. Deployment dan Demo

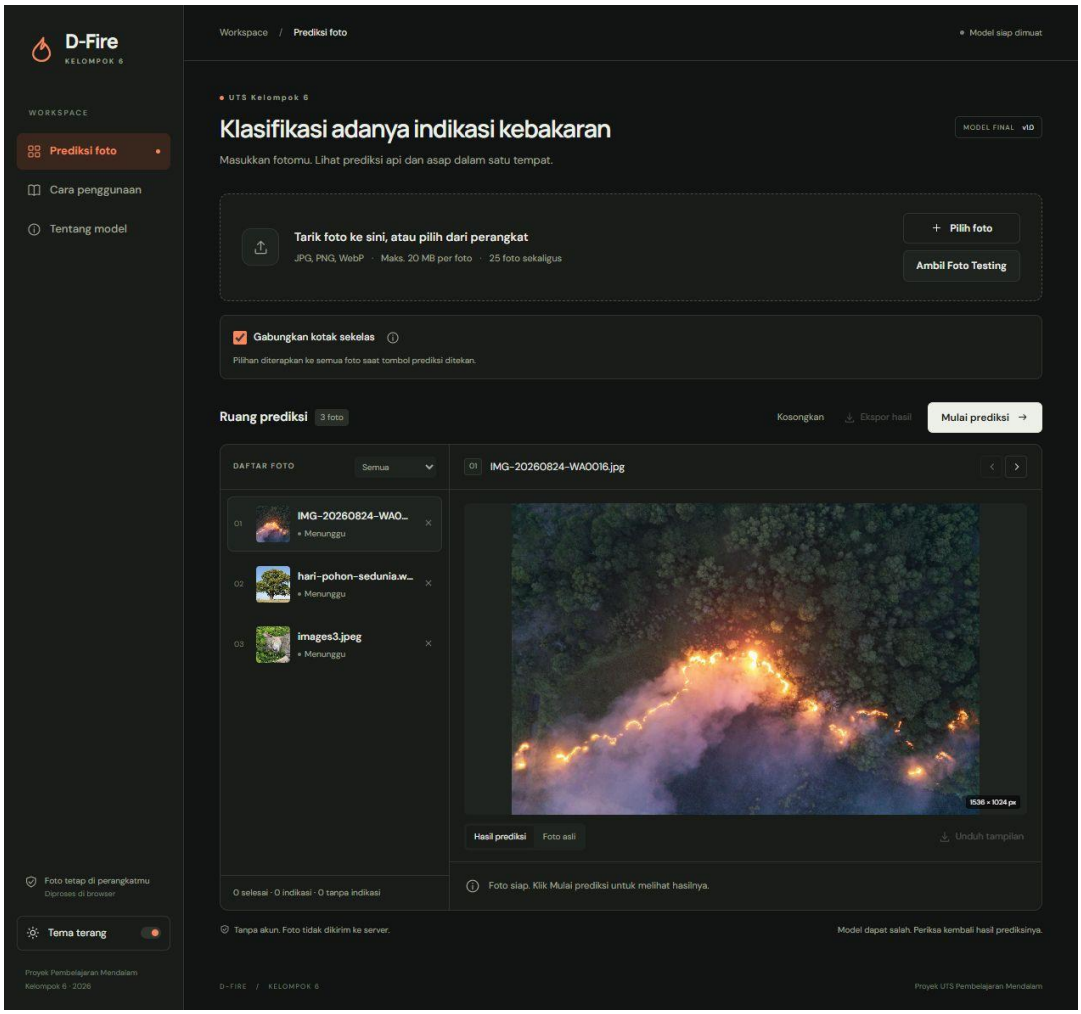
Alamat aplikasi dan repository

Aplikasi publik: https://klasifikasi-adanya-indikasi-kebakar.vercel.app/

Repository GitHub: https://github.com/noitsume/Deep-Learning-Kelompok-6

Tampilan antarmuka dan alur penggunaan

Halaman Prediksi foto menyediakan pemilihan foto, antrean, checkbox Gabungkan kotak sekelas, serta tombol Mulai prediksi. Tombol Ambil Foto Testing mengambil tiga gambar contoh ke antrean. Gambar baru dihitung setelah pengguna menekan tombol prediksi. Pada saat itu aplikasi membaca konfigurasi union dan memproses seluruh antrean secara berurutan.

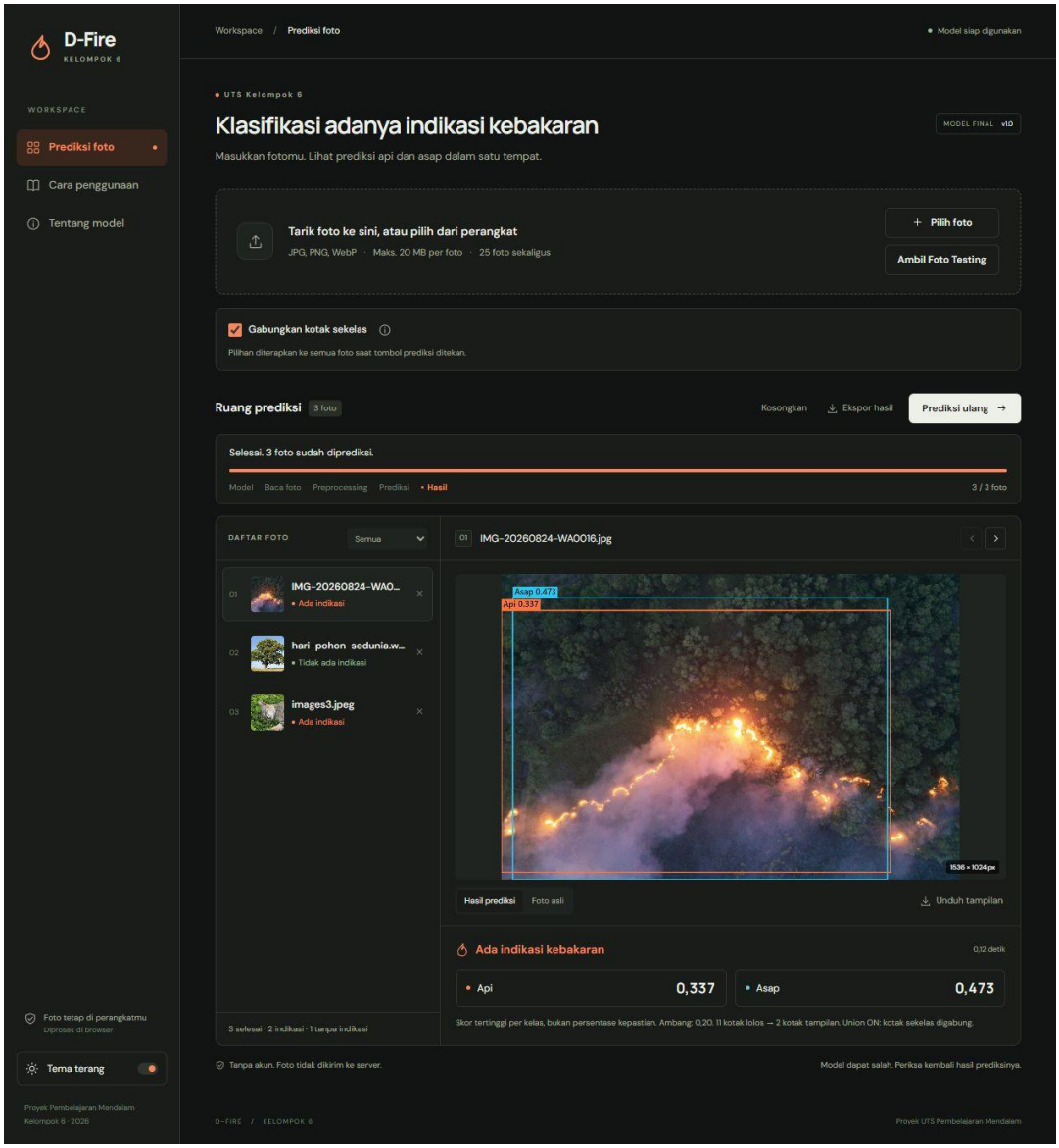


Gambar 10. Antarmuka web dengan tiga foto testing yang masih menunggu prediksi

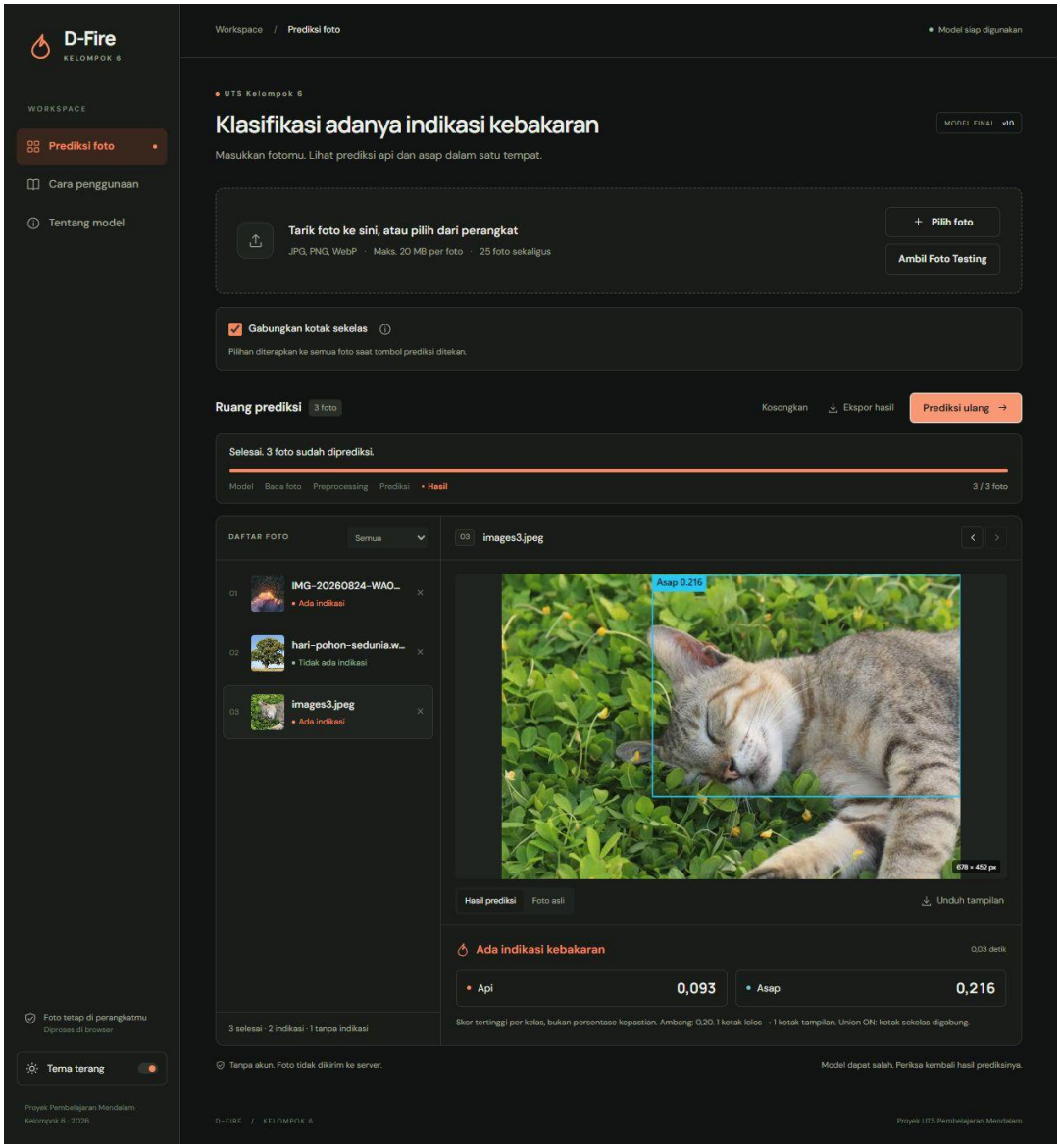
Sesudah proses selesai, setiap foto menampilkan label biner, skor tertinggi api dan asap, serta kotak hasil deteksi. Tab Foto asli dan Hasil prediksi memudahkan pemeriksaan visual. Unduh tampilan menyimpan PNG foto aktif; Ekspor hasil menyimpan JSON seluruh antrean.

Contoh masukan dan keluaran

Pengujian melalui alamat publik menggunakan tiga foto demonstrasi yang sama dengan notebook, terpisah dari 4.306 citra test. Foto kebakaran menghasilkan skor api 0,337 dan asap 0,473 dengan label ada indikasi; foto pohon memperoleh label tidak ada indikasi; foto kucing keliru dianggap mengandung asap dengan skor 0,216. Contoh terakhir merupakan false positive pada ambang 0,20.



Gambar 11. Prediksi benar pada foto kebakaran: api 0,337 dan asap 0,473

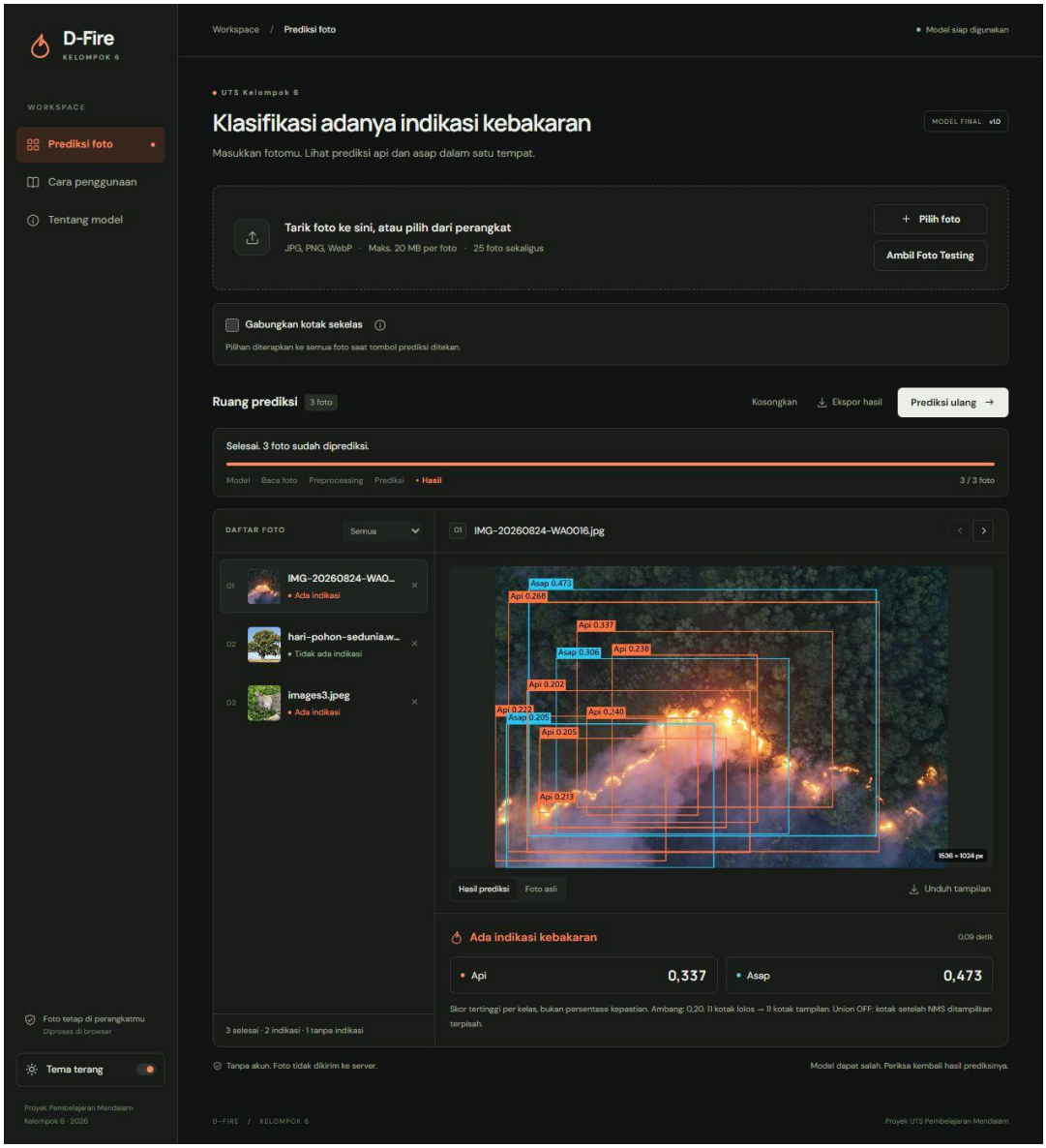


Gambar 12. Prediksi salah pada foto kucing: asap 0,216 melewati ambang 0,20

Penggabungan kotak sekelas

Checkbox Gabungkan kotak sekelas mengatur enclosing union untuk tampilan. Untuk tiap kelas, koordinat kiri dan atas memakai nilai minimum, sedangkan kanan dan bawah memakai nilai maksimum dari seluruh kotak yang lolos. Skor kotak gabungan diambil dari skor tertinggi dalam kelas tersebut. NMS dan label biner dihitung sebelum penggabungan.

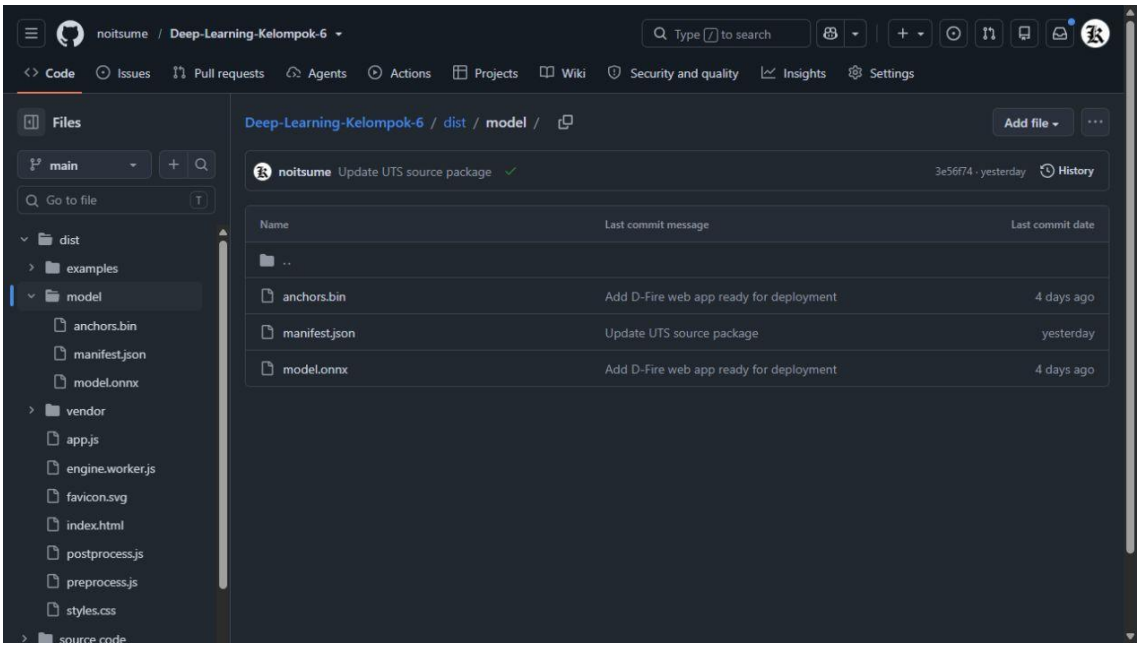
Pada foto kebakaran, union OFF menampilkan 11 kotak yang lolos; union ON merangkumnya menjadi dua kotak, masing-masing untuk asap dan api. Skor maksimum tetap 0,473 dan 0,337. Kotak gabungan dapat mencakup ruang di antara objek yang terpisah. Pengubahan checkbox diterapkan setelah Prediksi ulang ditekan.



Gambar 13. Union OFF menampilkan 11 kotak hasil penyaringan secara terpisah

Tahapan deployment

1. Menyimpan dan mengekspor model. Notebook menyimpan model.keras, anchors.npy, dan konfigurasi.json ke output/step7_model/. Skrip tools/export_model.py memuat model tersebut dan menghasilkan dist/model/model.onnx, anchors.bin, serta manifest.json. Manifest memuat ukuran masukan, urutan kelas, konfigurasi penyaringan, dan SHA-256 model. Keluaran ekspor dibandingkan dengan model Keras sebagaimana dijelaskan pada fitur inferensi ONNX.

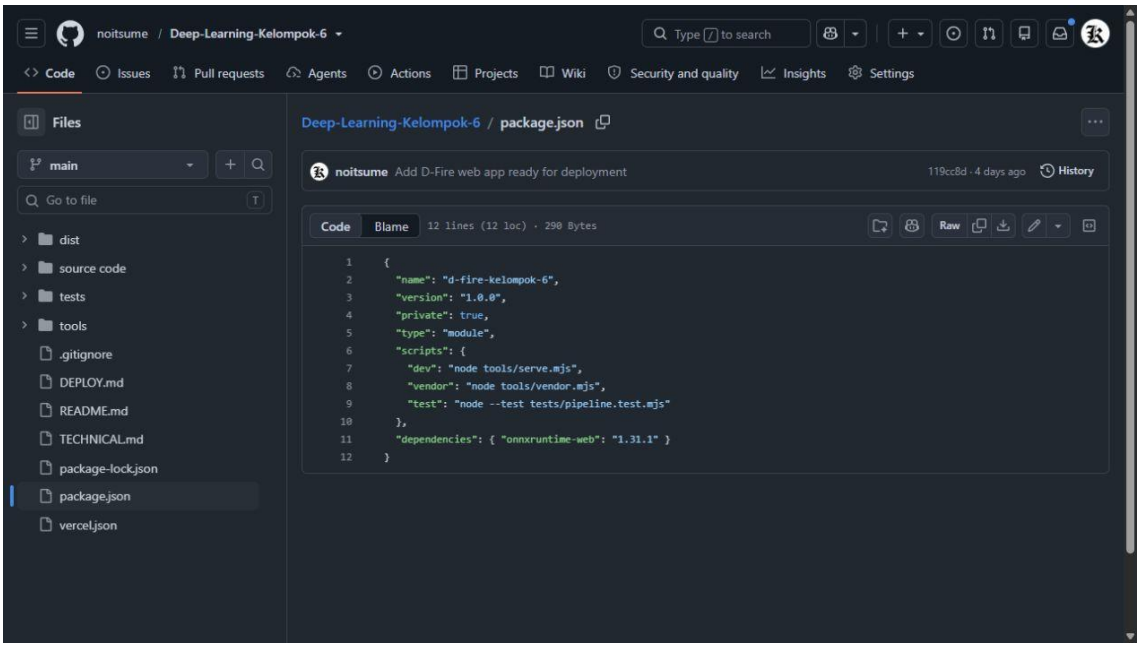


Gambar 14. Berkas model, anchor, dan manifest yang disertakan dalam repository

2. Menyiapkan skrip inferensi. engine.worker.js membaca foto, membentuk tensor [1, 128, 192, 3], lalu menjalankan session.run(). Hasil [1, 3456, 6] diteruskan ke decode() pada postprocess.js untuk sigmoid, dekode anchor, NMS, ambang 0,20, dan penggabungan tampilan. Preprocessing mengikuti ukuran serta rentang piksel masukan model notebook.

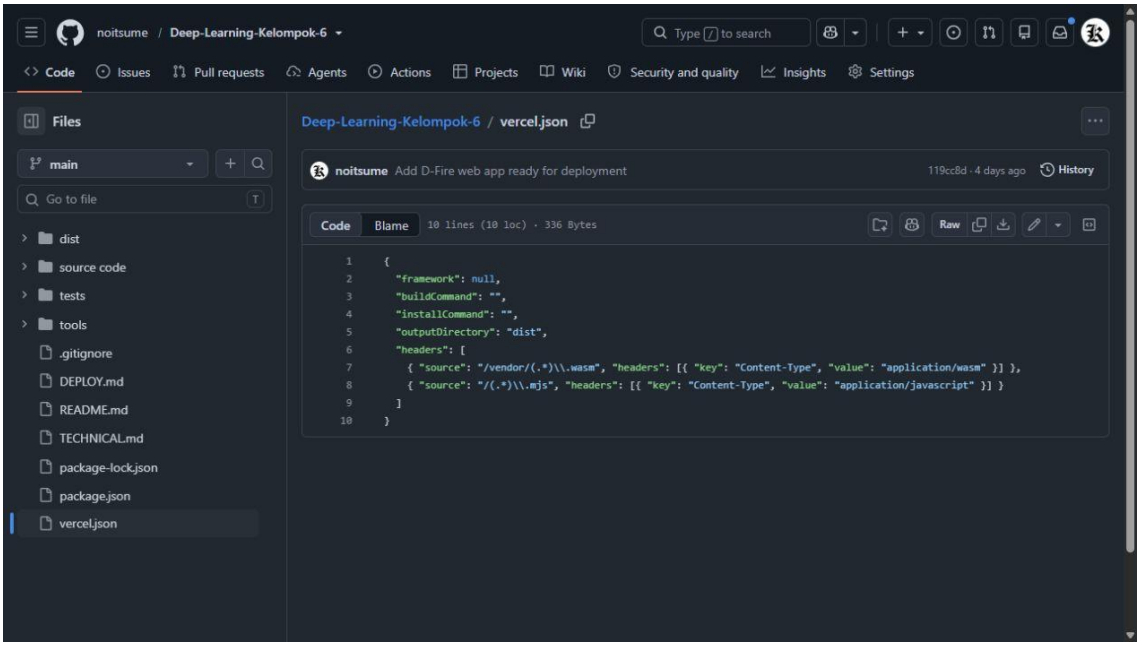
3. Menyusun antarmuka. index.html, styles.css, dan app.js menghubungkan masukan foto dengan Worker, menampilkan kemajuan proses, serta menyediakan hasil per foto. Pengujian antarmuka menggunakan foto kebakaran, pohon, dan kucing seperti pada gambar sebelumnya.

4. Menyiapkan dependensi. Paket onnxruntime-web dikunci pada versi 1.31.1 dalam package.json. Berkas JavaScript dan WebAssembly runtime disalin ke dist/vendor/ sehingga ikut disajikan bersama aplikasi. Pengembangan lokal dijalankan melalui npm run dev dan pemeriksaan fungsi melalui npm test.

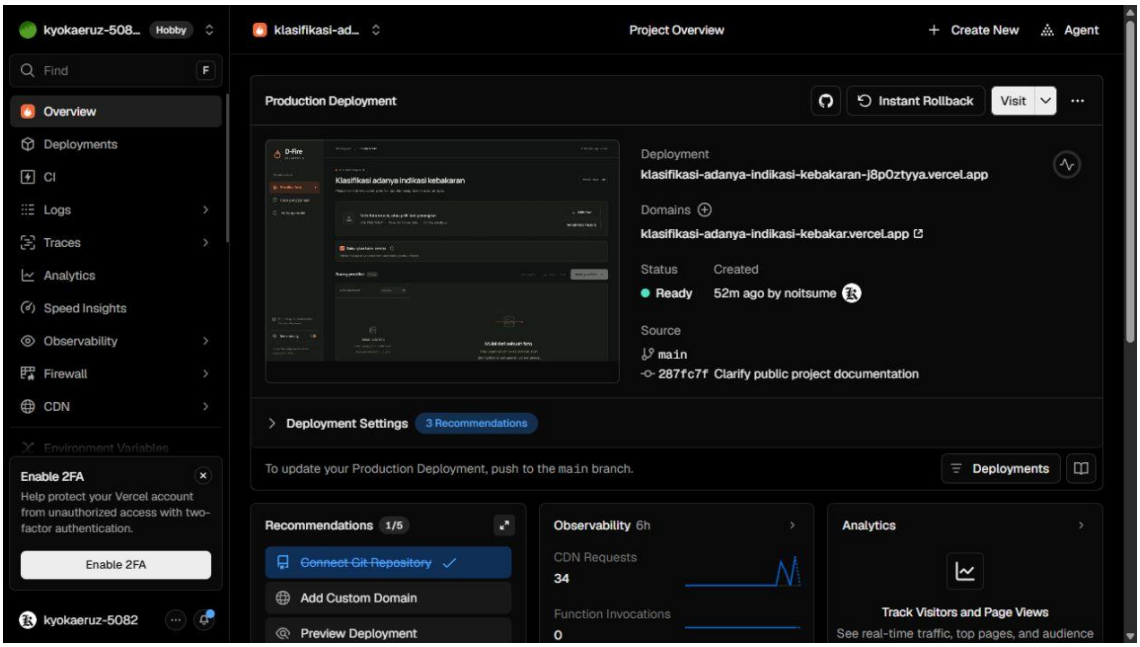


Gambar 15. Dependensi ONNX Runtime Web dan perintah pengembangan pada package.json

5. Menerbitkan aplikasi ke Vercel. Repository GitHub dihubungkan dengan proyek Vercel dan branch main digunakan untuk produksi. vercel.json menetapkan outputDirectory ke dist, dengan perintah build dan install kosong karena berkas publik sudah tersedia. Header menetapkan tipe application/wasm untuk .wasm dan application/javascript untuk .mjs.



Gambar 16. Konfigurasi direktori publik dan tipe berkas pada vercel.json



Gambar 17. Deployment produksi berstatus Ready dan terhubung ke branch main

6. Menguji alamat publik. Setelah status Ready, aplikasi dibuka melalui domain Vercel dan tombol Ambil Foto Testing digunakan. Ketiga foto masuk dengan status menunggu. Sesudah Mulai prediksi ditekan, hasil kebakaran, pohon, dan kucing sesuai hasil notebook. Pengujian union ON/OFF juga mempertahankan label serta skor, dengan jumlah kotak tampilan berubah sesuai pengaturan.

Video demo dan deployment

a. Antarmuka. Demonstrasi memperlihatkan halaman Prediksi foto, tombol Pilih foto dan Ambil Foto Testing, antrean gambar, konfigurasi union, serta tombol prediksi.

b. Masukan dan keluaran. Foto kebakaran dan pohon memperlihatkan prediksi yang benar, sedangkan foto kucing menunjukkan false positive. Skor dibaca bersama ambang 0,20, label biner, dan posisi kotak.

c. Dua fitur unik. Fitur ambang skor dijelaskan melalui keluaran api/asap dan perbandingan ambang pada laporan; aplikasi memakai nilai tetap 0,20. Inferensi ONNX diperlihatkan melalui tahap pemuatan model dan prediksi di browser, disertai penjelasan berkas model dan Worker. Union ON/OFF melengkapi demonstrasi penyajian kotak tanpa mengubah keputusan biner.

d. Deployment. Penjelasan mencakup model Keras yang diekspor ke ONNX, susunan berkas web dan dependensi, konfigurasi dist pada Vercel, status Ready, serta pengujian melalui alamat publik.

9. Lampiran

Cuplikan kode penting

Pembacaan citra menggunakan interpolasi bilinear. Nilai piksel dipertahankan pada rentang 0–255 hingga memasuki dua cabang model [6].

```
def baca_gambar(path):
    with Image.open(path) as im:
        rgb = im.convert('RGB').resize(
            (W, H), Image.Resampling.BILINEAR
        )
        return np.asarray(rgb, dtype=np.float32)
```

Pembentuk fitur kontras memisahkan piksel yang lebih terang dan lebih gelap daripada lingkungan sekitarnya pada dua ukuran jendela.

```
for k in [5, 15]:
    sekitar = layers.AveragePooling2D(
        k, strides=1, padding='same'
    )(luminans)
    beda = layers.Subtract()([luminans, sekitar])
    for arah in [1.0, -1.0]:
        z = layers.ReLU()(layers.Rescaling(arah*4)(beda))
        fitur.append(layers.AveragePooling2D(8)(z))
```

Kepala deteksi merangkai empat nilai regresi kotak dan dua logit kelas setiap anchor pada dimensi terakhir.

```
prediksi = layers.Concatenate(axis=-1)([
    layers.Reshape((-1, 4))(lokasi),
    layers.Reshape((-1, 2))(kelas)
])
```

Setelah NMS per kelas, fungsi prediksi memilih paling banyak 100 kandidat lalu menyaringnya dengan ambang skor.

```
hasil = sorted(
    hasil, key=lambda p: -p['skor']
)[:MAX_DETEKSI]
return [p for p in hasil if p['skor'] >= AMBANG]
```

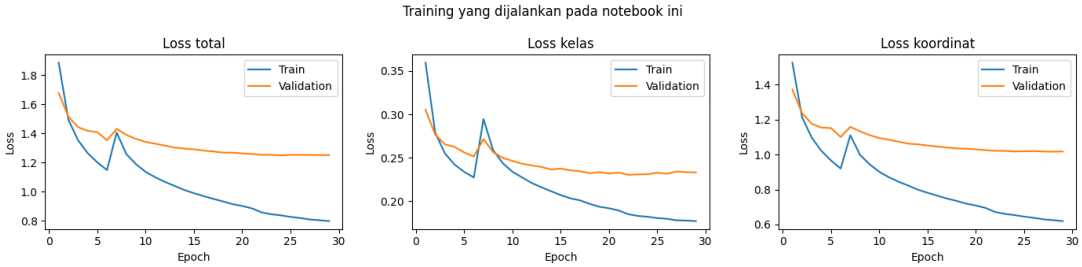
Keputusan per foto mengikuti keberadaan kotak yang lolos.

```
raw = model_prediksi(
    baca_gambar(path)[None], training=False
).numpy()[0]
kotak = prediksi_kotak(raw)
label = LABEL_BINER[int(bool(kotak))]
```

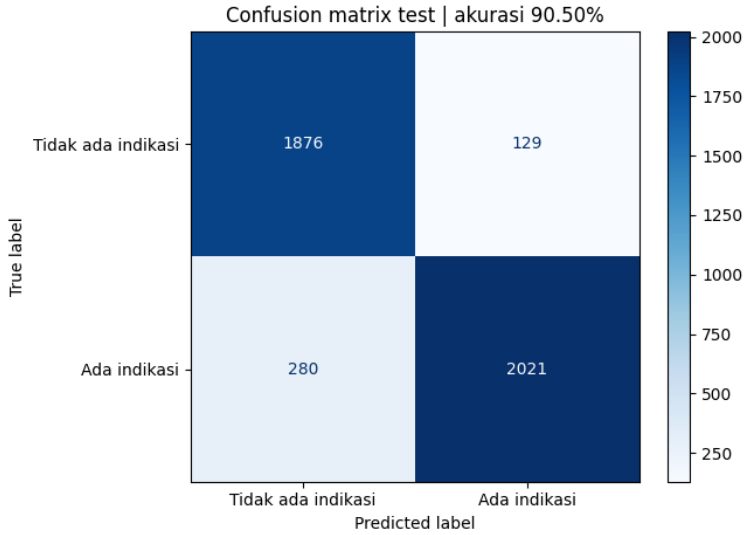
Pada web app, cuplikan berikut menjalankan model ONNX lalu meneruskan keluaran ke penyaringan kotak. mergeBoxes berasal dari konfigurasi checkbox yang dibaca ketika tombol prediksi ditekan [6].

```
const tensor = new ort.Tensor(
  'float32', input, [1, config.height, config.width, 3]
);
outputs = await session.run({
  [session.inputNames[0]]: tensor
});
const result = decode(
  outputs[session.outputNames[0]].data,
  anchors, config, mergeBoxes
);
```

Grafik evaluasi



Gambar 18. Kurva asli loss total, loss kelas, dan loss kotak pada notebook



Gambar 19. Matriks kebingungan asli pada 4.306 citra uji

Garis latih dan validasi pada Gambar 18 memperlihatkan perubahan galat selama pelatihan. Pada Gambar 19, baris adalah label acuan dan kolom adalah prediksi. Dua sel diagonal berjumlah 3.897 citra, sedangkan dua sel lainnya berjumlah 409 citra.

10. Daftar Pustaka

[1] de Venâncio, P. V. A. B., Lisboa, A. C., & Barbosa, A. V. (2022). An automatic fire detection system based on deep convolutional neural networks for low-power, resource-constrained devices. Neural Computing and Applications, 34, 15349–15368. Artikel jurnal

[2] Gaia, Solutions on Demand. D-Fire Dataset. Repositori resmi koleksi citra dan anotasi asap serta api. Repositori dataset

[3] Sayed Gamal. Smoke-Fire-Detection-YOLO. Kaggle, versi 1. Arsip yang dipakai oleh notebook kelompok. Dataset Kaggle

[4] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. Proceedings of CVPR, 4510–4520. Makalah MobileNetV2

[5] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal Loss for Dense Object Detection. Proceedings of ICCV, 2980–2988. Makalah focal loss

[6] Kelompok 6. (2026). UTS-Notebook.ipynb dan repository Deep-Learning-Kelompok-6. Notebook, hasil eksperimen, kode aplikasi, konfigurasi deployment, dan pemeriksaan konversi model. Repository proyek

[7] Keras. MobileNetV2. Dokumentasi implementasi backbone yang digunakan dalam proyek. Dokumentasi Keras

[8] ONNX. ONNX Concepts. Dokumentasi graf, operasi, parameter, dan bentuk masukan/keluaran model. Dokumentasi ONNX

[9] ONNX Runtime. How to add machine learning to your web application with ONNX Runtime. Dokumentasi inferensi pada browser. Dokumentasi ONNX Runtime Web

BAGIAN B. TAUTAN

1. Source code proyek

GitHub Kelompok 6:
https://github.com/noitsume/Deep-Learning-Kelompok-6

Paket notebook dan struktur output tersedia di folder source code, dalam berkas Source Code.zip. Kode aplikasi web dan model ONNX tersedia pada folder dist.

2. Video demo beserta tahapan deployment

Tautan video demo dan deployment belum dicantumkan.

3. Aplikasi yang sudah di-deploy

Aplikasi publik:
https://klasifikasi-adanya-indikasi-kebakar.vercel.app/